

实验报告：课后练习2（调试与性能分析）

项目	内容
课程名称	系统开发工具基础
实验主题	Debugging and Profiling（Missing Semester 2026 课后练习）
题目编号	课后练习2
学号	25020007191
姓名	周御基
实验日期	2026年8月31日

一、实验目的

1. 掌握调试器定位逻辑缺陷（pdb、gdb 反向调试）与内存错误检测工具（AddressSanitizer）
2. 学会用 strace 观察系统调用、用 LLM 辅助解析晦涩错误
3. 掌握性能剖析工具（perf stat / perf record / hyperfine）与资源监控、端口排查方法

二、实验环境

- 操作系统：Ubuntu Linux 25.04（远程服务器 192.168.79.130，VMware 虚拟机，4 CPU）
- 编译/调试：gcc 14.2.0、gdb 16.2、pdb（Python 3.13）、rr 5.9.0、AddressSanitizer
- 性能/监控：perf 6.14.11、hyperfine 1.19.0、htop、stress、taskset、ss、strace
- 远程连接：SSH（paramiko），部分工具经 sudo 安装

三、实验步骤与代码

本练习选自 Missing Semester 2026 第 4 讲「Debugging and Profiling」，共完成 10 道题（调试 5 题 + 性能 5 题）。

调试练习1：用调试器修复归并排序

实现含 bug 的归并排序 `merge_sort.py`，运行给定输入：

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] ≤ right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[i]); j += 1    # 缺陷行
    return result + left[i:] + right[j:]
```

```
python3 merge_sort.py
```

[1, 5, 4, 3, 4, 5, 6, 9] (期望 [1,1,2,3,4,5,6,9] · 结果错误)

使用 pdb 在缺陷行 (`right[i]`) 处设断点，观察变量：

```
p.cmdqueue.extend(["break /home/joe/practice2/merge_sort.py:8", "continue",
                  "p i", "p j", "p left[i]", "p right", "continue", ..., "quit"])
p.run("from merge_sort import merge; print('result=', merge([1,3],[1,4]))")
```

```
Breakpoint 1 at merge_sort.py:8
i = 1
j = 0
left[i] = 3
right = [1, 4]
result = [1, 4, 3, 4]
```

断点观察：命中 else 分支时 `i=1`、`j=0`，本应取 `right[j]=1`，但代码取 `right[i]=4`。**修复**：把 `right[i]` 改为 `right[j]`。修复后输出 [1, 1, 2, 3, 4, 5, 6, 9]，正确。

调试练习2：反向调试定位内存损坏

`corruption.c`： `curve_scores(0,5)` 循环 `i<4` 写 `scores[i]`，越界覆盖了相邻的 `students[1].id` (1002 变 1007)。

```
gcc -g corruption.c -o corruption && ./corruption
```

```
Student 1: id=1007
```

```
ERROR: Student 1's ID was corrupted! Expected 1002, got 1007 (rc=1)
```

使用 gdb 的软件记录反向调试 (`record` + `reverse-continue`) :

```
(gdb) record
(gdb) break corruption.c:40 ; continue
(gdb) p students[1].id      # 1007(已损坏)
(gdb) watch students[1].id
(gdb) reverse-continue      # 反向回退到覆盖 id 的那条写
```

```
Hardware watchpoint: students[1].id
Old value = 1007    New value = 1002
0x... in curve_scores (student_idx=0, curve=5) at corruption.c:24
24      students[student_idx].scores[i] += curve;
info locals → i = 3
```

根因: `curve_scores` 中 `for (int i=0; i<4; i++)` 使 `i=3` 时 `scores[3]` 越界, 写入与 `students[0].scores` 相邻的 `students[1].id`。修复: 循环改为 `i<3`。

补充: 本练习原计划使用 `rr` 反向调试。因远程为 VMware 虚拟机, **CPU 没有硬件性能计数器 (PMU)** (`perf stat` 显示 `cycles <not supported>`), `rr` 的 `perf_event_open` 无法工作 (`Unable to open performance counter`)。故改用 `gdb` 自带的软件记录反向调试, 达到相同的"运行到损坏处再回溯找元凶"效果。另需设置 `GLIBC_TUNABLES=...-AVX512F` 关闭 `glibc` 的 `AVX-512 strlen`, 否则 `gdb` 软件记录会碰到不支持的指令 (`Process record does not support instruction 0x62`) ——这也是练习5中交给 LLM 解析的晦涩错误之一。

调试练习3: AddressSanitizer

`uaf.c`: `free(greeting)` 之后还写 `greeting[0]='J'` (`use-after-free`)。

```
gcc uaf.c -o uaf && ./uaf      # 无 sanitizer
```

```
Hello, world!
```

```
J@>,      (乱码·看似没有问题, rc=0)
```

```
gcc -fsanitize=address -g uaf.c -o uaf_asan && ./uaf_asan
```

```

==57998==ERROR: AddressSanitizer: heap-use-after-free ... WRITE of size 1
    #0 ... uaf.c:12      ← greeting[0]='J'
freed by ... uaf.c:10    ← free(greeting)
previously allocated by ... uaf.c:6 ← malloc
Shadow bytes: ... [fd] ... (fd = Freed heap region)
SUMMARY: AddressSanitizer: heap-use-after-free ... (rc=1)

```

根因：在 `free(greeting)` 之后仍然对堆内存进行写操作。修复：把 `free(greeting)` 移动到最后一次使用之后。修复后用 ASan 重新编译运行，输出 `Hello, world! / Jello, world!`，`rc=0`，无报错。

调试练习4：strace 跟踪系统调用

```

strace -c ls -l / > /dev/null          # 汇总各系统调用次数/耗时
strace -e trace=opentat ls -l ~/practice2

```

```

% time ... calls errors syscall
...    25     0      statx
...    30    18     openat
...    20     0     listxattr
...    15     0     newfstatat
...     2     0     getdents64
...
opentat(AT_FDCWD, "/etc/ld.so.cache", ...) = 3
opentat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", ...) = 3
opentat(AT_FDCWD, "/usr/share/locale/en_US.UTF-8/LC_TIME/coreutils.mo", ...) = -1
ENOENT

```

分析：`ls -l` 先加载动态库（`ld.so.cache`、`libc.so.6` 等 `opentat / mmap`）；用 `getdents64` 读取目录项；`-l` 长格式对每个文件调用 `statx` 取元数据；`listxattr / lgetxattr` 用于取 ACL/颜色等扩展属性。`locale` 相关文件返回 `ENOENT` 属正常搜索回退。

调试练习5：用 LLM 解析晦涩错误

把调试过程中真实遇到的晦涩报错交给 LLM（智能体）解析并修复：

例1 — gdb 软件记录报错：

```

(gdb) record
Process record does not support instruction 0x62 at address 0x7ffff7da7f06.

```

LLM 解析: `0x62` 是 x86 **EVEX 前缀** (编码 AVX-512 指令)。glibc 的优化 `strlen` (`__strlen_evex`) 使用 AVX-512, 而 gdb 的软件记录 (`record full`) 不支持执行该指令簇。**修复**: 通过环境变量禁用 AVX-512 等新 SIMD 特性让 glibc 回退到可记录的标量路径:

```
set environment GLIBC_TUNABLES glibc.cpu.hwcaps=-AVX512F,-AVX2,-AVX
```

此后反向调试成功找到元凶 (见练习2)。

例2 — ASan 影子字节地图 (练习3 输出):

```
⇒0x ... : fa fa 00 00 00 fa fa fa[fd]fd fd fd ...  
Shadow byte legend: Addressable:00 Heap left redzone:fa Freed heap region:fd
```

LLM 解析: 目标地址处的影子字节为 `fd` (已释放的堆区), 说明该内存已被 `free`, 随后的写访问即 use-after-free。据此定位到 `uaf.c:12`。

说明: 这两个错误正是本练习中被 LLM 成功解释并给出修复的案例, 体现了"晦涩报错 → 交给 LLM 定位语义 → 修复"的调试 workflow。

性能练习1: perf stat 基本统计

```
perf stat ./corruption
```

```
Performance counter stats for './corruption':  
 2.94 msec task-clock          # 0.702 CPUs utilized  
 0      context-switches  
 61     page-faults  
<not supported> cycles
```

计数器含义:

- **task-clock / cpu-clock**: 进程实际占用 CPU 的时间 (毫秒); `CPUs utilized` 是运行窗口内占用的核数比例。
- **context-switches**: 进程先后被调度器切换进/出的次数 (0 次说明一次性跑完)。
- **cpu-migrations**: 进程在 CPU 核之间迁移的次数。
- **page-faults**: 进程触发的缺页/内存映射次数 (61 次为程序分配内存)。
- **cycles**: CPU 周期数 (硬件计数器)。本虚拟机无硬件 PMU, 故显示 `<not supported>`。

性能练习2: perf record + perf report

编译并采样分析 `slow.c` (大量 `sin / cos` 计算) :

```
gcc -g -O2 slow.c -o slow -lm
perf record -e task-clock -g ./slow      # 5935 samples 写入 perf.data
```

```
perf report --stdio
```

Children	Self	Command	Shared Object	Symbol
95.99%	3.47%	slow	slow	[.] main
			--- main	
			—50.72%—	__cos_fma
			--41.74%—	__sin_fma
52.60%	51.71%	slow	libm.so.6	[.] __cos_fma

分析: 时间几乎全部花在 `libm` 的 `__cos_fma` (52.6%) 与 `__sin_fma` (41.7%) 上——即 `slow_computation` 内层循环的 `sin / cos` 数学函数, 符合预期瓶颈。

火焰图: 本练习原本尝试用 `perf script | flamegraph.pl` 生成 SVG 火焰图, 但虚拟机**无外网** (`git clone https://github.com/...` 报 Connection reset), 无法下载 FlameGraph 脚本。改用 `perf report` 的调用图做等效的瓶颈可视化 (调用结构 `main → __sin_fma / __cos_fma` 与火焰图一致)。

性能练习3: hyperfine 基准测试

用 hyperfine 对比“统计文本词数”的两种实现 (随机 20 万词文件) :

```
hyperfine --warmup 2 'wc -w words2.txt' "awk '{n+=NF} END{print n}' words2.txt"
```

```
Benchmark 1: wc -w words2.txt
Time (mean ± σ):      6.3 ms ± 1.0 ms    [269 runs]
Benchmark 2: awk '{n+=NF} END{print n}' words2.txt
Time (mean ± σ):     157.1 ms ± 20.0 ms   [17 runs]
Summary
wc -w words2.txt ran
24.99 ± 5.10 times faster than awk ...
```

分析： `wc -w` 是 C 优化实现（整字段缓冲计数），比每行记录处理的 `awk` 快约 **25x**。hyperfine 自动多次运行取均值、标准差与极值，并给出带误差的加速比。

性能练习4：htop 监控 + taskset 限制 CPU

用 `taskset` 把 `stress -c 3` 限制到 2 个 CPU (0、2)，监控其负载与亲和性：

```
taskset --cpu-list 0,2 stress -c 3 &
ps -eo pid,psr,pcpu,comm | grep stress
taskset -pc <pid>
```

```
PID    PSR    %CPU   COMMAND
59576   2    87.5   stress
59577   0    52.8   stress
59578   2    55.7   stress
...
affinity list: 0,2          (所有 worker 都被限制在 CPU 0、2)
top -bn1: %Cpu(s): 53.6 us, ... 42.9 id
```

为什么 stress 不用 3 个 CPU？ `stress -c 3` 会派生 3 个 busy-wait 工作进程（都是单线程），但 `taskset --cpu-list 0,2` 把它们的 **CPU 亲和性全部限制到 0、2 两个核**。单线程进程同一时刻只能在一个核上运行，因此 3 个 worker 只能在 2 个核上被时间片轮转，`top` 里每个 worker 只占约 1/3 核、总负载停留在约 2 核水平（`idle 42.9%` 仍有空闲）。若不加限制，`-c 3` 在 4 核机器上会占满 3 核。

性能练习5：ss 查找占用端口并 kill

```
python3 -m http.server 4444 &
ss -tlnp | grep 4444
```

```
LISTEN 0 5 0.0.0.0:4444 0.0.0.0:*    users:(("python3",pid=59782,fd=3))
```

```
PID=$(ss -tlnp | grep 4444 | sed -n 's/.*pid=\([0-9]*\)*/\1/p')
ps -p 59782 -o pid,comm,args      # python3 -m http.server 4444
kill 59782
ss -tln | grep 4444 || echo "port 4444 is now free"
```

```
port 4444 is now free (no listener)
```

分析： `ss -tlnp` 对每个监听端口给出进程 `pid=`，用 `sed` 提取 PID 后可精确 `kill`，避免"端口被占"时手足无措。

四、实验结果

4.1 结果展示

编号	练习	关键结果
调试1	pdb 排序调试	断点发现 <code>right[i]</code> 应为 <code>right[j]</code> ，修复后正确排序
调试2	反向调试	<code>gdb reverse-continue</code> 回溯到 <code>curve_scores</code> 越界写 (<code>i=3</code>)
调试3	AddressSanitizer	检出 <code>heap-use-after-free</code> on <code>uaf.c:12</code>
调试4	strace	观察 <code>ls -l</code> 的 <code>openat/statx/getdents64</code> 等系统调用
调试5	LLM 解析错误	解释 <code>gdb 0x62</code> 指令与 ASan 影子字节，给出修复
性能1	perf stat	<code>task-clock/</code> 缺页等软件计数器可用， <code>cycles</code> 不受支持
性能2	perf record/report	热点为 <code>__cos_fma</code> / <code>__sin_fma</code>
性能3	hyperfine	<code>wc -w</code> 比 <code>awk</code> 快 24.99×
性能4	taskset 限制 CPU	<code>-c 3</code> 被限制在 {0,2} 两核，故不满 3 核
性能5	ss 找端口	定位 <code>pid=59782</code> 并 <code>kill</code> ，端口释放

4.2 结果分析

- 调试器价值：**断点/单步/观察变量可精确定位逻辑缺陷；`gdb` 的软件记录反向调试（`record + reverse-continue`）能在运行到损坏点后**回溯**找到覆盖内存的那一行，是排查内存损坏的有力手段（`rr` 因虚拟机无硬件 PMU 不可用，用 `gdb` 等价替代）。
- sanitizer 与静态工具：**AddressSanitizer 把不可见的 `use-after-free` 变成带调用栈与内存生命周期（分配→释放→误用）的明确报告，比裸跑出乱码可靠得多。
- 系统调用视角：**`strace` 揭示 `ls -l` 的库加载、目录读取、元数据统计等真实工作。
- 性能剖析闭环：**`perf stat`（宏观计数）→ `perf record/report`（热点函数）→ `hyperfine`（同任务两实现定量对比）层层深入；热点定位到 `libm` 的 `sin/cos`。
- 资源与进程管理：**`taskset` 限制亲和性后 `stress -c 3` 只能占用被允许的 2 核；`ss -tlnp + kill` 是解决"端口被占用"的标准手段。
- 环境限制：**虚拟机 (VMware) 无硬件 PMU (`perf cycles`、`rr` 受限)、无外网（火焰图脚本无法下载），均已用可用的替代方案完成并说明。

五、实验总结

通过本次实验，我掌握了以下技能：

1. 用 pdb 断点观察变量定位算法缺陷，并最小化修复
 2. 用 gdb 软件记录反向调试定位内存越界写（替代环境受限的 rr）
 3. 用 AddressSanitizer 检测 use-after-free 等内存错误
 4. 用 strace 观察系统调用，用 LLM 辅助解析晦涩编译/调试报错
 5. 用 perf stat / perf record+report 做程序性能剖析，定位热点函数
 6. 用 hyperfine 对同任务不同实现做定量基准
 7. 用 htop/top + taskset 监控并限制 CPU，用 `ss -tlnp` + `kill` 排查占用端口的进程
-

实验中的脚本文件：

1. `Scripts/practice2/merge_sort.py`、`Scripts/practice2/pdb_drive.py`
2. `Scripts/practice2/corruption.c`、`Scripts/practice2/rr_gdb.gdb`
3. `Scripts/practice2/uaf.c`、`Scripts/practice2/uaf_fixed.c`
4. `Scripts/practice2/obscure.c`
5. `Scripts/practice2/slow.c`